NOVUZ INC.

Software Engineer — Take-Home Project

Interview project

novuz.com

Role	Python / ML Engineer
Time Limit	24 hours (self-timed, honor system)
Submission	GitHub repo link + written design doc (PDF or MD)
Stack	Python 3.11+ | NumPy allowed for array ops only | No vector DB libraries

Overview

You will design and implement a tree-structured vector retrieval system from first principles, then wire it to a live LLM API to answer user queries using retrieved context. This project is intentionally open-ended: there is no single correct solution. We are evaluating how you think, how you structure systems under constraints, and how clearly you communicate trade-offs.

Read this entire document before writing any code.

Background & Motivation

CMaaP (Continuous Modernization as a Product) is Novuz's SaaS platform for enterprise data warehouse migration. It handles complex SQL dialect translation (T-SQL, Teradata, SSIS) and must answer user questions accurately while enforcing guardrails around out-of-scope topics.

A naive flat vector store does not scale for a multi-tenant platform where different customers have different domain contexts. This project asks you to build a hierarchical retrieval system that routes queries through a metadata-aware tree before performing vector similarity search at the leaf level.

System Description

1. The Routing Tree

Design and implement a multi-children tree where:

- Each internal node contains metadata describing its domain scope (e.g., { product: 'cmaap', topic: 'sql-migration', dialect: 'tsql' })
- Given an incoming query, the tree is traversed by matching query metadata against node metadata
- If no child matches, the node follows a default path to the next level
- Leaf nodes are reached when no further routing is possible or the match is definitive

The tree must support:

- Arbitrary branching factor (not binary)
- Recursive traversal with a well-defined base case
- Graceful fallback — a query that matches nothing should still reach a leaf via the default path

You may hard-code a sample tree for demonstration. The structure should be instantiated programmatically (not read from a file), so the grader can modify and re-run it easily.

2. Leaf Node: Dual Vector Stores

Each leaf node contains two independent vector stores:

FAQ Store

Contains question-answer pairs relevant to that leaf's domain. Each entry is a vector representation of the FAQ question text.

Guardrail Store

Contains vectors representing topics or queries that are out-of-scope or should be blocked. If the query is sufficiently close to a guardrail vector, the system must refuse to answer and return a safe response instead.

3. Retrieval Algorithm

Given a query Q and a target k, retrieve exactly k FAQs from the leaf's FAQ store using the following algorithm:

Step 1 — Vectorize the query

Produce a vector representation of Q. You may use fasttext, and numpy. Install them in your virtual environment "pip install fasttext numpy". Document your choice if you pick any other vector representation and its limitations. To use fasttext: here is a code snippet

```
import fasttext
import fasttext.util
fasttext.util.download_model('en', if_exists='ignore')
```

```
ft = fasttext.load_model('cc.en.300.bin')
query_vec = ft.get_sentence_vector("Why generation fails for 'THOW'")
```

Step 2 — Guardrail check

Compute cosine similarity between selected fat vectors and every guardrail vector. If any similarity exceeds a configurable threshold θ_guard, remove the faq from the list and pick the next ranking faq.from the FAQ store. Assume all the FAQ vectors are 1_guard, away from any guardrail vector. θ_guard = 1_guard + pick_threshold, where pick_threshold is minimal cosine similarity between query and FAQ vector that can be picked

Step 3 — Initial ranking

Compute cosine similarity between Q and every FAQ vector. Rank FAQs descending by score. Take the top k as your candidate set C. cosine similarity has to be greater than pick_threshold

Step 4 — Similarity collapse filter

For each FAQ f in C: if cosine_similarity(Q, f) > θ_collapse, that FAQ is considered too similar to the query itself (likely a verbatim match or trivially redundant). Remove f from C and replace it with the next highest-ranked FAQ not already in C. Repeat until |C| = k or the FAQ store is exhausted.

Step 5 — LLM call

Format the k FAQs as context. Just print results, no actual call is needed. (Display only)

4. Vector & Cosine Implementation

You must implement cosine similarity from scratch. NumPy is allowed for array arithmetic. Do not import sklearn, scipy, or any library with a cosine_similarity function.

Expected signature:

```
def cosine_similarity(a: list[float], b: list[float]) -> float: ...
```

Your implementation should handle edge cases: zero vectors, mismatched dimensions (raise clearly), floating point precision.

Deliverables

Submit a GitHub repository containing:

- **tree.py:** RoutingTree, TreeNode, LeafNode classes. Full recursive traversal logic.
- **vector_store.py:** VectorStore class with add(), query(k), cosine_similarity(). No library shortcuts.
- **retriever.py:** Retrieval pipeline: vectorize → guardrail check → rank → collapse filter → LLM call.
- **llm_client.py:** Thin wrapper for Claude or OpenAI API. API key via environment variable.
- **main.py:** Runnable demo. Instantiates a sample tree, adds FAQs and guardrails, runs 3 sample queries.
- **DESIGN.md:** Written design document (see below).
- **requirements.txt:** All dependencies pinned.

Design Document Requirements

DESIGN.md must address the following. Bullet points are fine; depth matters more than length.

Complexity Analysis

- Time complexity of a single query end-to-end: tree traversal + guardrail check + FAQ ranking + collapse filter. Express as O() with justification.
- Space complexity of the tree and vector stores as a function of number of nodes N, FAQs per leaf F, vector dimension D.
- Where are the bottlenecks? What would you profile first under load?

Scaling Proposal

Describe how you would evolve this system to handle:

- 10,000 leaf nodes each with 50,000 FAQs
- 1,000 concurrent queries per second
- Multi-tenant isolation (different customers must never share context)

Address at minimum: sharding strategy, approximate nearest neighbor (ANN) alternatives, caching, and whether the routing tree structure helps or hurts at scale.

Collapse Filter Design

- Justify your choice of θ_collapse. How would you tune it in production?
- What happens when k > |FAQ store| after filtering? How does your implementation handle this?
- Is the collapse filter applied before or after guardrail check? Does order matter?

Guardrail Limitations

- What adversarial inputs could bypass a cosine-similarity-based guardrail?

- How would you make the guardrail more robust?

Starter Code

The following scaffold is provided to save setup time. You are not required to use it, but the interfaces should remain compatible.

tree.py

```
from __future__ import annotations

from dataclasses import dataclass, field

from typing import Any, Optional



@dataclass

class TreeNode:

    metadata: dict[str, Any]

    children: list[TreeNode] = field(default_factory=list)

    default_child: Optional[TreeNode] = None

    leaf: Optional[LeafNode] = None



    def is_leaf(self) -> bool:

        return self.leaf is not None



    def route(self, query_meta: dict[str, Any]) -> TreeNode:

        # TODO: implement routing logic

        raise NotImplementedError
```

```
@dataclass
class LeafNode:
    faq_store: VectorStore       # import from vector_store.py
    guardrail_store: VectorStore


class RoutingTree:
    def __init__(self, root: TreeNode): ...
    def traverse(self, query_meta: dict) -> LeafNode: ...
```

vector_store.py

```
class VectorStore:
    def __init__(self, name: str):
        fasttext.util.download_model('en', if_exists='ignore')
        self.ft = fasttext.load_model('cc.en.300.bin')



    def add(self, text: str, payload: dict) -> None:
        """Vectorize text and store with payload."""
        raise NotImplementedError


    def query(self, text: str, k: int) -> list[dict]:
        """Return top-k results sorted by cosine similarity desc."""
        qvec = self.ft.get_sentence_vector(text)
        ….
```

```
    @staticmethod
    def cosine_similarity(a: list[float], b: list[float]) -> float:
        raise NotImplementedError
```

retriever.py

```
class Retriever:
    def __init__(self, tree: RoutingTree, k: int = 5,
                 theta_guard: float = 0.85,
                 theta_collapse: float = 0.95): ...

    def retrieve(self, query: str, query_meta: dict) -> list[dict]:
        """
        Full pipeline: route -> guardrail -> rank -> collapse filter.
        Returns k FAQ payloads or raises GuardrailException.
        """
        raise NotImplementedError
```

Evaluation Rubric

Area	Weight	What we look for
Correctness	**25%**	Tree traversal works. Guardrail fires correctly. Collapse filter produces exactly k results. LLM is called with proper context.
Recursion & Tree Design	**20%**	Traversal is genuinely recursive (not iterative). Base case is clean. Default path is handled elegantly. No globals.

Vector & Cosine Impl	**15%**	Cosine from scratch, correct formula, edge cases handled. Vectorization explained and justified.
Complexity Analysis	**20%**	O() is derived correctly for the full pipeline. Space complexity addressed. Bottlenecks identified honestly.
Scale Proposal	**10%**	Practical, specific. Shows understanding of ANN, sharding, multi-tenancy. Not hand-wavy.
Code Quality	**10%**	Readable, typed, modular. Would you be comfortable reviewing this person's PRs?

Constraints & Rules

Allowed

- Python standard library (collections, math, itertools, heapq, etc.)
- NumPy for array arithmetic only (dot product, norm — not cosine_similarity)
- requests or httpx for LLM API calls
- dataclasses, typing, abc

Not Allowed

- sklearn, scipy, faiss, chromadb, pinecone, weaviate, or any vector database
- sentence-transformers, openai.embeddings, or any pre-built embedding function
- LangChain, LlamaIndex, or any RAG framework
- Iterative tree traversal (must be recursive)

Gray Areas — Document Your Choice

- Approximate cosine using bit manipulation or SIMD tricks — allowed if exact version also present

Bonus Challenges

These are not required but will meaningfully differentiate strong candidates:

- **HNSW sketch:** Sketch (not full implementation) of how you'd replace linear scan with a Hierarchical Navigable Small World index. Include complexity comparison.
- **Multi-query fan-out:** Support querying multiple leaf nodes in parallel when the routing tree is ambiguous and merging/re-ranking results.
- **Persistence:** Serialize and deserialize the full tree + vector stores to disk without pickle.
- **Evaluation harness:** A script that measures precision@k against a small labeled test set you define.

Submission Instructions

- Push to a public GitHub repo named rag-tree-<your-github-handle>
- README.md must include: setup instructions, how to run main.py, sample output, and your LLM provider choice
- Email your repo link to ted@novuz.com subhajit@novuz.com deb@novuz.com with subject: RAG Tree Submission — [Your Name]

Questions? Email subhajit@novuz.com. We respond within 4 hours on business days.

We're not looking for perfection — we're looking for engineers who think clearly, communicate honestly, and build systems they can defend.

Good luck.